

TALLER SEMANA 9 — SQL, QUERY BUILDER Y REPORTES DEL CRM

Proyecto Integrador CRM "Gestión Integral de Negocios" — ALP-365

Docente: Ing. José Daniel Cadenas L. — I.U.J.O. Extensión Barquisimeto **Fechas:** Martes 30 y Miércoles 31 de septiembre de 2026 **Duración:** 4h 15min (2 sesiones) **Evaluación:** 15% (5% grupal + 10% individual con defensa y PDF)



CONTENIDO

1. Inicio
 2. Parte I — Conceptos SQL (de básico a avanzado)
 3. Parte II — Montaje del CRM en Laravel
 4. Parte III — Los 2 reportes con razonamiento
 5. Parte IV — Plantilla PDF de entrega
 6. Parte V — Rúbrica y sistema de preguntas por cédula
-

PARTE I — CONCEPTOS SQL (COMPLETO Y COMPACTO)



MÓDULO 1 — SQL BÁSICO

1.1 ¿Qué es SQL?

Es el lenguaje estándar para comunicarse con bases de datos relacionales (MySQL, PostgreSQL, etc.). Es como el "español" que usamos para pedirle datos a la base de datos.

1.2 SELECT — Traer datos

```
-- Traer TODO
SELECT * FROM clients;

-- Traer columnas específicas (más eficiente)
SELECT nombre_empresa, zona_geografica FROM clients;
```

1.3 WHERE — Filtrar

```
SELECT * FROM clients WHERE zona_geografica = 'Oeste';
SELECT * FROM clients WHERE user_id = 2;
```

```
SELECT * FROM clients WHERE nombre_empresa LIKE '%Lara%';
```

Operadores útiles: =, <>, >, <, >=, <=, LIKE, IN, BETWEEN

1.4 ORDER BY — Ordenar

```
SELECT * FROM clients ORDER BY nombre_empresa ASC;    -- A-Z
SELECT * FROM clients ORDER BY created_at DESC;        -- Más recientes primero
```

1.5 COUNT() — Contar

```
SELECT COUNT(*) AS total FROM clients;
SELECT COUNT(*) AS total FROM clients WHERE zona_geografica = 'Este';
```

1.6 GROUP BY — Agrupar

```
-- ¿Cuántos clientes hay en CADA zona?
SELECT zona_geografica, COUNT(*) AS total
FROM clients
GROUP BY zona_geografica;
```

💡 **Regla de oro:** Cuando usas **GROUP BY**, en el **SELECT** solo puedes poner la columna por la que agrupas y funciones de agregación (**COUNT**, **SUM**, **AVG**, **MAX**, **MIN**).

🤔 PREGUNTAS DE COMPRENSIÓN 1 (responde en tu bitácora)

P1. Si cambio **WHERE zona_geografica = 'Oeste'** por **WHERE zona_geografica LIKE '%Este%'**, ¿qué clientes aparecerán?

P2. ¿Qué pasaría si quito el **GROUP BY** y dejo solo **SELECT zona_geografica, COUNT(*)**? ¿Funcionaría? ¿Por qué?

🎯 MÓDULO 2 — SQL INTERMEDIO (JOINS)

2.1 ¿Por qué necesitamos JOINS?

En el CRM, los datos están relacionados:

- Un cliente **pertenece** a un asesor (**user_id** → **users.id**)
- Un cliente **tiene muchas** interacciones (**interactions.client_id** → **clients.id**)

Para mostrar información combinada, necesitamos unir tablas.

2.2 INNER JOIN — Solo los que coinciden

```
SELECT clients.nombre_empresa, users.name AS asesor
FROM clients
INNER JOIN users ON clients.user_id = users.id;
```

⚠ Si un cliente no tiene asesor, **NO aparece**.

2.3 LEFT JOIN — Todos los de la izquierda + coincidencias

```
SELECT users.name AS asesor, COUNT(clients.id) AS total
FROM users
LEFT JOIN clients ON clients.user_id = users.id
GROUP BY users.id, users.name;
```

✅ Si un asesor no tiene clientes, aparece con `total = 0`.

2.4 JOIN múltiple — Tres tablas

```
SELECT clients.nombre_empresa, users.name AS asesor, origins.nombre AS origen
FROM clients
INNER JOIN users ON clients.user_id = users.id
INNER JOIN origins ON clients.origin_id = origins.id;
```

🤔 PREGUNTAS DE COMPRENSIÓN 2

P3. Tenemos 3 asesores: Ana (5 clientes), Carlos (0 clientes), María (3 clientes). ¿Qué mostrará un `INNER JOIN` para Carlos? ¿Y un `LEFT JOIN`?

P4. En el reporte de interacciones por asesor, ¿por qué usamos `LEFT JOIN` y no `INNER JOIN`?

🎯 MÓDULO 3 — QUERY BUILDER DE LARAVEL

3.1 ¿Qué es?

Es la forma **segura y elegante** de hacer consultas SQL en Laravel, sin escribir SQL crudo. Protege contra inyección SQL automáticamente.

3.2 Equivalencias SQL ↔ Query Builder

SQL	Query Builder
<code>SELECT * FROM clients</code>	<code>DB::table('clients')->get()</code>
<code>WHERE zona = 'Oeste'</code>	<code>->where('zona_geografica', 'Oeste')</code>
<code>ORDER BY nombre</code>	<code>->orderBy('nombre_empresa')</code>
<code>GROUP BY zona</code>	<code>->groupBy('zona_geografica')</code>

SQL	Query Builder
COUNT(*)	DB::raw('COUNT(*) as total')
LEFT JOIN	->leftJoin('users', ...)

3.3 Ejemplo completo

SQL:

```
SELECT zona_geografica, COUNT(*) AS total
FROM clients
GROUP BY zona_geografica
ORDER BY total DESC;
```

Query Builder:

```
$zonas = DB::table('clients')
    ->select('zona_geografica', DB::raw('COUNT(*) as total'))
    ->groupBy('zona_geografica')
    ->orderByDesc('total')
    ->get();
```

3.4 Collections de Laravel

Cuando usas `->get()`, Laravel devuelve una **Collection** con métodos útiles:

```
$totalGeneral = $zonas->sum('total'); // Sumar
$labels = $zonas->pluck('zona_geografica')->toArray(); // Extraer columna
$zonas->map(function($z) { ... }); // Transformar
$zonas->filter(function($z) { ... }); // Filtrar
```

🤔 PREGUNTAS DE COMPRENSIÓN 3

P5. ¿Qué hace `DB::raw()` y por qué debemos usarlo con cuidado?

P6. ¿Qué diferencia hay entre `->get()` y `->first()`?

📦 PARTE II — MONTAJE DEL CRM

🎯 FASE 0 — CREAR EL PROYECTO (20 min)

Paso 0.1 — Crear el proyecto

```
cd C:\xampp\htdocs
composer create-project laravel/laravel crm-gestion-integral "12.*"
```

```
cd crm-gestion-integral
```

Paso 0.2 — Crear la base de datos en phpMyAdmin

- Nombre: `crm_gestion_integral`
- Cotejamiento: `utf8mb4_unicode_ci`

Paso 0.3 — Configurar `.env`

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=crm_gestion_integral
DB_USERNAME=root
DB_PASSWORD=
```

Paso 0.4 — Crear migraciones

```
php artisan make:model Client -m
php artisan make:migration create_users_simple_table
php artisan make:migration create_interactions_table
php artisan make:migration create_origins_table
php artisan migrate
```

Paso 0.5 — Migración de `clients`

```
public function up(): void
{
    Schema::create('clients', function (Blueprint $table) {
        $table->id();
        $table->string('nombre_empresa', 150);
        $table->string('contacto_principal', 100);
        $table->string('telefono_whatsapp', 20);
        $table->enum('zona_geografica', ['Oeste', 'Este', 'Cabudare', 'Centro',
'Zona Industrial']);
        $table->unsignedBigInteger('user_id')->nullable();
        $table->unsignedBigInteger('origin_id')->nullable();
        $table->timestamps();
    });
}
```

Paso 0.6 — Migración de `users`

```
public function up(): void
{
    Schema::create('users', function (Blueprint $table) {
        $table->id();
```

```

        $table->string('name');
        $table->string('email')->unique();
        $table->timestamps();
    });
}

```

Paso 0.7 — Migración de `interactions`

```

public function up(): void
{
    Schema::create('interactions', function (Blueprint $table) {
        $table->id();
        $table->foreignId('client_id')->constrained()->onDelete('cascade');
        $table->enum('tipo_interaccion', ['Llamada', 'Visita', 'WhatsApp']);
        $table->text('observaciones')->nullable();
        $table->date('fecha_seguimiento');
        $table->timestamps();
    });
}

```

Paso 0.8 — Migración de `origins`

```

public function up(): void
{
    Schema::create('origins', function (Blueprint $table) {
        $table->id();
        $table->string('nombre', 100);
        $table->timestamps();
    });
}

```

Paso 0.9 — Modelo `Client` con `$fillable`

Archivo: `app/Models/Client.php`

```

<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Client extends Model
{
    protected $fillable = [
        'nombre_empresa', 'contacto_principal', 'telefono_whatsapp',
        'zona_geografica', 'user_id', 'origin_id',
    ];
}

```

FASE 1 — SEEDER CON DATOS DE PRUEBA (15 min)

Paso 1.1 — Crear el seeder

```
php artisan make:seeder CrmDemoSeeder
php artisan db:seed --class=CrmDemoSeeder
```

Paso 1.2 — Código del seeder `database/seeder/CrmDemoSeeder.php`

```
<?php
namespace Database\Seeders;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;

class CrmDemoSeeder extends Seeder
{
    public function run(): void
    {
        // 1. Asesores
        foreach (['Ana Rodríguez', 'Carlos Pérez', 'María González'] as $name) {
            DB::table('users')->insert([
                'name' => $name,
                'email' => strtolower(str_replace(' ', '.', $name)) . '@crm.com',
                'created_at' => now(), 'updated_at' => now(),
            ]);
        }

        // 2. Orígenes
        foreach (['Redes Sociales', 'Recomendación', 'Web', 'Evento', 'Otro'] as
$nombre) {
            DB::table('origins')->insert([
                'nombre' => $nombre, 'created_at' => now(), 'updated_at' => now(),
            ]);
        }

        // 3. Clientes
        $zonas = ['Oeste', 'Este', 'Cabudare', 'Centro', 'Zona Industrial'];
        $empresas = [
            'Distribuidora Lara C.A.', 'Tecnoservicios Barquisimeto',
            'Ferretería El Tornillo', 'Panificadora La Espiga',
            'Auto Repuestos Central', 'Farmacia Salud Total',
            'Comercial Los Andes', 'Servicios Técnicos Zulia',
            'Inversiones Barquisimeto', 'Textiles del Centro',
            'Alimentos del Este', 'Logística Express Lara',
            'Constructora Occidente', 'Repuestos Cabudare', 'Mini Market 24'
        ];
        foreach ($empresas as $i => $empresa) {
            DB::table('clients')->insert([
                'nombre_empresa' => $empresa,
                'contacto_principal' => 'Contacto ' . ($i + 1),
                'telefono_whatsapp' => '58414' . str_pad(rand(1000000, 9999999), 7,
```

```

'0', STR_PAD_LEFT),
    'zona_geografica' => $zonas[$i % 5],
    'user_id' => ($i % 3) + 1,
    'origin_id' => ($i % 5) + 1,
    'created_at' => now(), 'updated_at' => now(),
]);
}

// 4. Interacciones
$tipos = ['Llamada', 'Visita', 'WhatsApp'];
foreach (DB::table('clients')->get() as $client) {
    foreach (range(1, rand(3, 6)) as $j) {
        DB::table('interactions')->insert([
            'client_id' => $client->id,
            'tipo_interaccion' => $tipos[array_rand($tipos)],
            'observaciones' => 'Seguimiento #' . $j,
            'fecha_seguimiento' => now()->subDays(rand(1, 60)),
            'created_at' => now(), 'updated_at' => now(),
        ]);
    }
}
$this->command->info('✅ CRM poblado.');
```

◆ PARTE III — LOS 2 REPORTES CON RAZONAMIENTO

🎯 FASE 2 — CONTROLADOR Y RUTAS

```
php artisan make:controller ReportController
```

En `routes/web.php`:

```

use App\Http\Controllers\ReportController;

Route::prefix('reportes')->name('reportes.')->group(function () {
    Route::get('/zonas', [ReportController::class, 'clientesPorZona'])->name('zonas');
    Route::get('/interacciones', [ReportController::class, 'interaccionesPorAsesor'])->name('interacciones');
});
```

📊 REPORTE 1 — CLIENTES POR ZONA (con huecos)

🤖 PREDICCIÓN ANTES DE CODIFICAR (responde en tu bitácora)

- ¿Qué tabla necesitamos consultar? _____
- ¿Por cuál columna debemos agrupar? _____
- ¿Qué función de agregación necesitamos? _____
- ¿En qué orden queremos los resultados? _____

Paso 3.1 — Método con HUECOS (el estudiante completa)

```
<?php
namespace App\Http\Controllers;
use Illuminate\Support\Facades\DB;

class ReportController extends Controller
{
    public function clientesPorZona()
    {
        // 1. Consulta con Query Builder
        $zonas = DB::table('_____') // ← HUECO A
            ->select(
                '_____', // ← HUECO B
                DB::raw('_____ as total') // ← HUECO C
            )
            ->groupBy('_____') // ← HUECO D
            ->orderByDesc('_____') // ← HUECO E
            ->_____(); // ← HUECO F

        // 2. Total general
        $totalGeneral = $zonas->_____('total'); // ← HUECO G

        // 3. Agregar porcentaje
        $zonasConPorcentaje = $zonas->_____(function ($zona) use ($totalGeneral) {
// ← HUECO H
            $zona->porcentaje = $totalGeneral > 0
                ? round((_____ / $totalGeneral) * 100, 2) // ← HUECO I
                : 0;
            return _____; // ← HUECO J
        });

        // 4. Datos para gráfico
        $labels = $zonasConPorcentaje->_____('zona_geografica')->toArray(); // ←
HUECO K
        $data = $zonasConPorcentaje->_____('total')->toArray(); // ←
HUECO L

        return view('reports.zonas', compact(
            'zonasConPorcentaje', 'totalGeneral', 'labels', 'data'
        ));
    }
}
```

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Clientes por Zona</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
<div class="container mt-4">
    <h1 class="mb-4"><img alt="Flag of Mexico" data-bbox="305 288 318 301"/> Clientes por Zona Geográfica</h1>

    <div class="row mb-4">
        <div class="col-md-6">
            <div class="card text-bg-primary">
                <div class="card-body">
                    <h6>Total de Clientes</h6>
                    <h2>{{ $totalGeneral }}</h2>
                </div>
            </div>
        </div>
        <div class="col-md-6">
            <div class="card text-bg-success">
                <div class="card-body">
                    <h6>Zonas Activas</h6>
                    <h2>{{ $zonasConPorcentaje->count() }}</h2>
                </div>
            </div>
        </div>
    </div>

    <div class="card mb-4">
        <div class="card-header bg-dark text-white"><h5 class="mb-0">Detalle por
Zona</h5></div>
        <div class="card-body">
            <table class="table table-striped">
                <thead class="table-dark">
                    <tr><th>#</th><th>Zona</th><th>Total</th><th>%</th><th>Barra
visual</th></tr>
                </thead>
                <tbody>
                    @foreach($zonasConPorcentaje as $i => $zona)
                        <tr>
                            <td>{{ $i + 1 }}</td>
                            <td><strong>{{ $zona->zona_geografica }}</strong></td>
                            <td><span class="badge bg-primary fs-6">{{ $zona->total
}}</span></td>
                            <td>{{ $zona->porcentaje }}%</td>
                            <td>
                                <div class="progress" style="height: 20px;">

```

```

<div class="progress-bar bg-success"
style="width: {{ $zona->porcentaje }}%">
    {{ $zona->porcentaje }}%
</div>
</div>
</td>
</tr>
@endforeach
</tbody>
</table>
</div>
</div>

<div class="card">
    <div class="card-body"><canvas id="grafico" height="80"></canvas></div>
</div>

<div class="mt-3">
    <a href="{{ route('reportes.interacciones') }}" class="btn btn-
primary">Siguiendo reporte →</a>
</div>
</div>

<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js">
</script>
<script>
    new Chart(document.getElementById('grafico'), {
        type: 'bar',
        data: {
            labels: @json($labels),
            datasets: [{
                label: 'Clientes por Zona',
                data: @json($data),
                backgroundColor: ['#0d6efd', '#198754', '#ffc107', '#dc3545',
'#6f42c1']
            }]
        },
        options: { responsive: true, scales: { y: { beginAtZero: true } } }
    });
</script>
</body>
</html>

```

RETO DE MODIFICACIÓN (obligatorio)

Modifica el reporte para que muestre **solo las zonas con más del 15% del total**. Pista: usa `>filter()` de Collection.

REPORTE 2 — INTERACCIONES POR ASESOR (con huecos)

PREDICCIÓN ANTES DE CODIFICAR

- ¿Qué tipo de JOIN necesitamos: INNER o LEFT? _____
- ¿Por cuál tabla debemos empezar la consulta? _____
- ¿Por qué? _____

Paso 4.1 — Método con HUECOS

```
public function interaccionesPorAsesor()
{
    $asesores = DB::table('_____') // ← HUECO A
        ->select(
            'users.id', 'users.name',
            DB::raw("COUNT(CASE WHEN interactions.tipo_interaccion = '_____'
THEN 1 END) AS llamadas"), // ← HUECO B
            DB::raw("COUNT(CASE WHEN interactions.tipo_interaccion = '_____'
THEN 1 END) AS visitas"), // ← HUECO C
            DB::raw("COUNT(CASE WHEN interactions.tipo_interaccion = '_____'
THEN 1 END) AS whatsapp"), // ← HUECO D
            DB::raw('COUNT(_____.id) AS total') // ← HUECO E
        )
        ->_____Join('clients', 'clients.user_id', '=', 'users.id') //
← HUECO F
        ->_____Join('interactions', 'interactions.client_id', '=',
'clients.id') // ← HUECO G
        ->groupBy('_____', '_____') // ← HUECO H, I
        ->orderByDesc('_____') // ← HUECO J
        ->get();

    $labels = $asesores->_____('name')->toArray(); // ← HUECO K
    $llamadas = $asesores->_____('llamadas')->toArray(); // ← HUECO L
    $visitas = $asesores->_____('visitas')->toArray(); // ← HUECO M
    $whatsapp = $asesores->_____('whatsapp')->toArray(); // ← HUECO N

    return view('reports.interacciones', compact(
        'asesores', 'labels', 'llamadas', 'visitas', 'whatsapp'
    ));
}
}
```

Paso 4.2 — Vista `resources/views/reports/interacciones.blade.php`

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Interacciones por Asesor</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
<div class="container mt-4">
```

```

<h1 class="mb-4">📞 Interacciones por Asesor</h1>

<div class="card mb-4">
  <div class="card-header bg-dark text-white"><h5 class="mb-0">Desglose por
Tipo</h5></div>
  <div class="card-body">
    <table class="table table-hover">
      <thead class="table-dark">
        <tr>
          <th>#</th><th>Asesor</th>
          <th class="text-center">📞 Llamadas</th>
          <th class="text-center">📅 Visitas</th>
          <th class="text-center">💬 WhatsApp</th>
          <th class="text-center">TOTAL</th>
        </tr>
      </thead>
      <tbody>
        @foreach($asesores as $i => $a)
          <tr>
            <td>{{ $i + 1 }}</td>
            <td><strong>{{ $a->name }}</strong></td>
            <td class="text-center"><span class="badge bg-primary">
{{ $a->llamadas }}</span></td>
            <td class="text-center"><span class="badge bg-success">
{{ $a->visitas }}</span></td>
            <td class="text-center"><span class="badge bg-warning
text-dark">{{ $a->whatsapp }}</span></td>
            <td class="text-center"><span class="badge bg-danger fs-
6">{{ $a->total }}</span></td>
          </tr>
        @endforeach
      </tbody>
    </table>
  </div>
</div>

<div class="card">
  <div class="card-body"><canvas id="grafico" height="80"></canvas></div>
</div>

<div class="mt-3">
  <a href="{{ route('reportes.zonas') }}" class="btn btn-secondary">←
Anterior</a>
</div>
</div>

<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js">
</script>
<script>
  new Chart(document.getElementById('grafico'), {
    type: 'bar',
    data: {
      labels: @json($labels),
      datasets: [

```

```

        { label: 'Llamadas', data: @json($llamadas), backgroundColor:
'#0d6efd' },
        { label: 'Visitas', data: @json($visitas), backgroundColor:
'#198754' },
        { label: 'WhatsApp', data: @json($whatsapp), backgroundColor:
'#ffc107' }
    ]
  },
  options: {
    responsive: true,
    scales: { x: { stacked: true }, y: { stacked: true, beginAtZero: true }
  }
}
});
</script>
</body>
</html>

```

RETO DE MODIFICACIÓN (obligatorio)

Agrega un desglose adicional: muestra cuántas interacciones se hicieron **por día de la semana** (Lunes, Martes, etc.). Pista: usa `DAYNAME(fecha_seguimiento)` en MySQL.

PARTE IV — PLANTILLA PDF DE ENTREGA

Instrucciones: Copie esta plantilla en Word/Google Docs, complétela y expórtela a PDF. Máximo 4 páginas. Suba el PDF a GitHub.

INICIO DE LA PLANTILLA (copiar desde aquí)

INFORME TÉCNICO — TALLER SEMANA 9

CRM "Gestión Integral de Negocios" — ALP-365

Docente: Ing. José Daniel Cadenas L. **Fecha:** __/09/2026

Datos del estudiante

- **Nombre y apellido:** _____
- **Cédula:** V-_____ (IMPORTANTE: con esta cédula se asignan las preguntas de la defensa)
- **Sección:** ____ (A / B)
- **Grupo N°:** ____
- **Correo institucional:** _____
- **Repositorio GitHub:** <https://github.com/>_____

Asignación de preguntas de defensa

Los 3 últimos dígitos de mi cédula son: ____ La suma de estos dígitos es: ____ El resultado de (suma % 10) es: ____ **Mi SET de preguntas asignado es: SET ____** (letra A–J)

✓ EVIDENCIA 1 — Capturas de los 2 reportes (2%)

Reporte 1 — Clientes por Zona: [INSERTAR CAPTURA]

Reporte 2 — Interacciones por Asesor: [INSERTAR CAPTURA]

✓ EVIDENCIA 2 — Respuestas a las 6 preguntas de comprensión (3%)

P1. _____ R:

P2. _____ R:

P3. _____ R:

P4. _____ R:

P5. _____ R:

P6. _____ R:

✓ EVIDENCIA 3 — Código comentado de UN método (2%)

Copie el método de UNO de los reportes y coméntelo línea por línea:

```
// [PEGAR CÓDIGO COMENTADO]
```

✓ EVIDENCIA 4 — Retos de modificación resueltos (2%)

Reto 1 (filtro > 15%):

```
// [PEGAR CÓDIGO]
```

Reto 2 (desglose por día):

```
// [PEGAR CÓDIGO]
```

✓ EVIDENCIA 5 — Reflexión breve (100 palabras)

1. ¿Qué concepto fue más difícil?
2. ¿Cómo aplicarás esto al CRM final?

FIN DE LA PLANTILLA

◆ PARTE V — RÚBRICA Y SISTEMA DE PREGUNTAS POR CÉDULA

📋 RÚBRICA (15%)

A. Evaluación Grupal (5%)

Criterio	5 pts	3 pts	1 pt
Funcionalidad	Los 2 reportes funcionan	1 reporte funciona	Ninguno funciona
Retos	Resolvió ambos retos	Resolvió 1 reto	No resolvió
Interfaz	Bootstrap + Chart.js bien integrados	Aceptable	Básico

B. Evaluación Individual (10%)

Componente	Puntos	Criterio rápido de corrección
PDF completo (6 evidencias)	4 pts	¿Tiene las 6 secciones completas?
Defensa oral (3 preguntas del SET asignado)	6 pts	2 pts por pregunta

💡 Guía rápida de corrección del PDF (para el docente)

Sección	¿Qué revisar?	Puntos
Evidencia 1	¿Hay 2 capturas claras con URL visible?	2
Evidencia 2	¿Las 6 respuestas son coherentes (no copiadas)?	3
Evidencia 3	¿El código está comentado con sentido?	2
Evidencia 4	¿Los retos están resueltos y funcionan?	2
Evidencia 5	¿La reflexión tiene mínimo 100 palabras?	1
Evidencia 6	¿Hay commit en GitHub?	0
TOTAL PDF		10

Defensa oral: 3 preguntas del SET asignado = 6 pts adicionales.

Cómo calcular el SET asignado

1. Tomar los **3 últimos dígitos** de la cédula.
2. **Sumar** esos 3 dígitos.
3. Calcular el **módulo 10** de esa suma (el residuo de dividir entre 10).
4. El resultado (0 a 9) indica la letra del SET:

Resultado	SET
0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H
8	I
9	J

Ejemplos

Ejemplo 1: Cédula V-12.345.**678**

- Dígitos: $6 + 7 + 8 = 21$
- $21 \% 10 = 1 \rightarrow$ **SET B**

Ejemplo 2: Cédula V-25.678.**123**

- Dígitos: $1 + 2 + 3 = 6$
- $6 \% 10 = 6 \rightarrow$ **SET G**

Ejemplo 3: Cédula V-19.876.**550**

- Dígitos: $5 + 5 + 0 = 10$
- $10 \% 10 = 0 \rightarrow$ **SET A**

BANCO DE 10 SETS DE PREGUNTAS (3 preguntas por SET)

Cada SET contiene 3 preguntas de diferentes niveles para evaluar comprensión integral.

SET A

1. ¿Qué hace `SELECT * FROM clients?` ¿Y si cambio `*` por `nombre_empresa?`
2. ¿Cuál es la diferencia entre `INNER JOIN` y `LEFT JOIN`?

3. ¿Qué hace `DB::raw()` y por qué debemos usarlo con cuidado?

◆ SET B

1. ¿Para qué sirve `WHERE`? Dame un ejemplo del CRM.
2. Si un asesor no tiene clientes, ¿aparece en el reporte con `LEFT JOIN`? ¿Por qué?
3. ¿Qué diferencia hay entre `->get()` y `->first()`?

◆ SET C

1. ¿Qué diferencia hay entre `ORDER BY ASC` y `DESC`?
2. ¿Por qué usamos `LEFT JOIN` en el reporte de interacciones por asesor?
3. ¿Cómo previene Laravel la inyección SQL en el Query Builder?

◆ SET D

1. ¿Qué hace `COUNT(*)`? ¿Cuándo lo usarías en el CRM?
2. ¿Qué pasaría si usamos `INNER JOIN` en lugar de `LEFT JOIN` en el reporte 2?
3. ¿Qué es una **Collection** de Laravel? Menciona 2 métodos que usaste.

◆ SET E

1. ¿Por qué necesitamos `GROUP BY` cuando usamos `COUNT()`?
2. En el reporte de interacciones por asesor, ¿por qué agrupamos por `users.id` Y `users.name`?
3. Si agregamos una nueva zona geográfica, ¿el reporte se actualiza solo? ¿Por qué?

◆ SET F

1. ¿Qué pasaría si quito el `GROUP BY` y dejo solo `SELECT zona_geografica, COUNT(*)`?
2. ¿Qué hace `COUNT(CASE WHEN ... THEN 1 END)`? ¿Por qué es mejor que hacer 3 consultas separadas?
3. ¿Cómo agregarías un filtro por rango de fechas al reporte de productividad?

◆ SET G

1. ¿Qué hace `DB::table('clients')`?
2. Si un asesor no tiene clientes, ¿aparece en el reporte? ¿Por qué?
3. ¿Qué harías si el gráfico de Chart.js no se muestra en el navegador?

◆ SET H

1. Explica con tus palabras qué es una función de agregación. Da 2 ejemplos.
2. ¿Por qué `->pluck('name')` es mejor que hacer un `foreach` para extraer nombres?
3. Si tuvieras que agregar un filtro por rango de fechas, ¿dónde lo pondrías y por qué?

◆ SET I

1. ¿Qué diferencia hay entre `WHERE zona = 'Oeste'` y `WHERE zona LIKE '%Oeste%'`?
2. ¿Qué hace `->orderByDesc('total')`? ¿Cómo lo escribirías en SQL crudo?
3. Si la consulta tardara 10 segundos con 100,000 clientes, ¿qué optimizarías?

◆ SET J

1. ¿Qué significa `nullable()` en una migración? ¿Por qué `user_id` es nullable en el CRM?
2. ¿Qué es `@json($labels)` en Blade y por qué no usamos `{{ $labels }}`?
3. Explica con tus palabras la diferencia entre Eloquent ORM y Query Builder.



ANEXO — ESTRUCTURA FINAL DE ARCHIVOS

```
crm-gestion-integral/
├── app/Http/Controllers/
│   └── ReportController.php           ← NUEVO
├── app/Models/
│   └── Client.php                     ← NUEVO
├── database/
│   ├── migrations/
│   │   ├── xxxx_create_clients_table.php
│   │   ├── xxxx_create_users_simple_table.php
│   │   ├── xxxx_create_interactions_table.php
│   │   └── xxxx_create_origins_table.php
│   └── seeders/
│       └── CrmDemoSeeder.php          ← NUEVO
├── resources/views/
│   └── reports/
│       ├── zonas.blade.php           ← NUEVO
│       └── interacciones.blade.php    ← NUEVO
└── routes/web.php                     ← MODIFICADO
```